



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/RAI/TYBTECH /SEM-V/DS/2026-27

Experiment No. : 4

Title: Time Series Analysis for Predicting Robotic Sensor Failures

Batch:A2
Experiment No.: 4

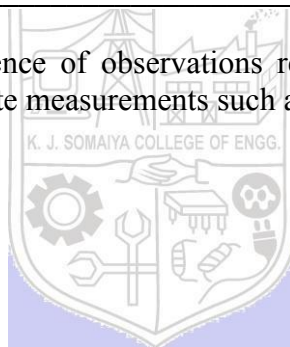
Roll No.:16014324028

Aim: To perform time series analysis on robotic sensor data, identify temporal trends and abnormal sensor behavior, and predict potential robotic sensor failures using time-dependent features and a machine learning model.

Resources needed: Computer/Laptop, Python 3.x, Jupyter Notebook / Google Colab Kaggle sensor dataset containing time stamped robotic/IoT sensor readings, Python libraries

Theory :- A **time series** is a sequence of observations recorded in chronological order. In robotics, sensors continuously generate measurements such as (Example):

- Temperature
- Vibration
- Pressure
- Motor current
- Speed
- Torque
- Humidity
- Voltage



Time series analysis examines how sensor values change with time. Important characteristics include:

- **Trend:** Long-term increase or decrease in sensor values.
- **Seasonality:** Repeating patterns at regular intervals.
- **Variation:** Normal fluctuations in sensor measurements.
- **Anomaly:** An unusual observation that differs significantly from normal behavior.

In robotic systems, a gradual increase in vibration or temperature may indicate deterioration of a component.

A moving average is used to smooth short-term fluctuations and identify the underlying trend. For a window of n observations:

$$MA_t = \frac{X_t + X_{t-1} + \dots + X_{t-n+1}}{n}$$

A moving average of sensor vibration can help identify whether vibration is gradually increasing before a failure.

Failure prediction attempts to identify whether a sensor/system is likely to experience a failure based on historical observations.

Time-dependent features can include:

- Current sensor value
- Previous sensor value
- Moving average
- Moving standard deviation
- Change in sensor value
- Temperature trend
- Vibration trend

These features can be provided to a machine learning model to classify observations as **Normal** or **Failure**.

Activity:

Step 1: Obtain the Dataset :- Download a suitable **Kaggle robotic/IoT sensor dataset** containing:

- a) Timestamp
- b) One or more sensor measurements
- c) Failure/anomaly/status information

Step 2: Import Required Libraries

Step 3: Load the Dataset

Step 4: Convert Timestamp and Sort Data

Step 5: Check Missing Values

Step 6: Visualize Sensor Values Over Time

Step 7: Calculate Moving Average

Step 8: Calculate Rolling Standard Deviation

Step 9: Detect Abnormal Sensor Behavior

Step 10: Create Time-Series Features

Step 11: Prepare Data for Failure Prediction

Step 12: Split Data into Training and Testing Sets

Step 13: Train Failure Prediction Model

Step 14: Evaluate the Model

Step 15: Interpret the Results

Students should answer:

1. Which sensor showed the strongest temporal variation?
2. Were abnormal vibration values observed before failures?
3. Did the moving average reveal a clear trend?
4. What percentage of observations were identified as anomalies?
5. How accurately did the model identify failures?
6. Which sensor features contributed most to failure prediction?

Results: (program with output)**Code:-**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, confusion_matrix, classification_report
DATA =
"/media/galdrux/galdrux_storage/sem_5/DS/dataset/EV_Battery_Time_Series_D
ataset.csv"
df = pd.read_csv(DATA)
print("Dataset loaded successfully.")
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
df = df.sort_values("Timestamp")
df = df.reset_index(drop=True)
print("\nTimestamp converted and data sorted chronologically.")
sensor_cols =
["SOC_percent", "Voltage_V", "Current_A", "Temperature_C", "Vibration_g", "Cap
acity_Ah"]
print("\n" + "=" * 65)
print("RESULT 1 – DATASET INFORMATION")
print("=" * 65)
print("Dataset Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())
print("\nData Types:")
print(df.dtypes)
print("\nMissing Values Before Preprocessing:")
print(df[sensor_cols].isna().sum())
df[sensor_cols] =
df[sensor_cols].interpolate(method="linear").bfill().ffill()
print("\nMissing values handled using linear interpolation.")
print("\nMissing Values After Preprocessing:")
print(df[sensor_cols].isna().sum())
print("\nFailure Distribution:")
print(df["Failure"].value_counts().sort_index())
print("\nInjected Anomaly Distribution:")
print(df["Anomaly"].value_counts().sort_index())
```

```
plt.figure(figsize=(12,5))
plt.plot(df["Timestamp"],df["Temperature_C"],label="Temperature (°C)")
plt.plot(df["Timestamp"],df["Voltage_V"],label="Voltage (V)")
plt.title("EV Battery Sensor Values Over Time")
plt.xlabel("Timestamp")
plt.ylabel("Sensor Value")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

window = 30
df["Vibration_MA"] = df["Vibration_g"].rolling(window).mean()
df["Vibration_Rolling_STD"] = df["Vibration_g"].rolling(window).std()
plt.figure(figsize=(12,5))
plt.plot(df["Timestamp"],df["Vibration_g"],alpha=0.4,label="Vibration")
plt.plot(df["Timestamp"],df["Vibration_MA"],linewidth=2,label="30-Sample
Moving Average")
plt.title("EV Battery Vibration with Moving Average")
plt.xlabel("Timestamp")
plt.ylabel("Vibration (g)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

rolling_mean = df["Vibration_g"].rolling(window,min_periods=10).mean()
rolling_std = df["Vibration_g"].rolling(window,min_periods=10).std()
df["Vibration_ZScore"] =
(df["Vibration_g"]-rolling_mean)/rolling_std.replace(0,np.nan)
df["Detected_Anomaly"] = (df["Vibration_ZScore"].abs()>3).astype(int)
anomaly_count = df["Detected_Anomaly"].sum()
anomaly_percentage = (anomaly_count/len(df))*100
print("\n" + "=" * 65)
print("RESULT 4 – ANOMALY DETECTION")
print("=" * 65)
print("Detected Anomalies:",anomaly_count)
print(f"Percentage of Observations Identified as Anomalies:
{anomaly_percentage:.2f}%")
plt.figure(figsize=(12,5))
plt.plot(df["Timestamp"],df["Vibration_g"],label="Vibration")
anomaly_points = df["Detected_Anomaly"] == 1
```

```
plt.scatter(df.loc[anomaly_points,"Timestamp"],df.loc[anomaly_points,"Vib
ration_g"],s=25,label="Detected Anomaly")
plt.title("EV Battery Vibration Anomaly Detection")
plt.xlabel("Timestamp")
plt.ylabel("Vibration (g) ")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
for col in sensor_cols:
    df[f"{col}_Previous"] = df[col].shift(1)
    df[f"{col}_Diff"] = df[col].diff()
    df[f"{col}_MA"] = df[col].rolling(window).mean()
    df[f"{col}_STD"] = df[col].rolling(window).std()
df = df.replace([np.inf,-np.inf],np.nan)
df = df.bfill().ffill()
features = []
for col in sensor_cols:
    features.extend([col,f"{col}_Previous",f"{col}_Diff",f"{col}_MA",f"{col}_
STD"])
X = df[features]
y = df["Failure"]
print("\nNumber of ML Features:",len(features))
split = int(len(df)*0.80)
X_train = X.iloc[:split]
X_test = X.iloc[split:]
y_train = y.iloc[:split]
y_test = y.iloc[split:]
print("\nTraining Samples:",len(X_train))
print("Testing Samples :",len(X_test))
model = RandomForestClassifier(n_estimators=150,max_depth=10,class_weight="balanc
ed",random_state=42)
model.fit(X_train,y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test,y_pred)
precision = precision_score(y_test,y_pred,zero_division=0)
recall = recall_score(y_test,y_pred,zero_division=0)
f1 = f1_score(y_test,y_pred,zero_division=0)
```

```
cm = confusion_matrix(y_test,y_pred)
print("\n" + "=" * 65)
print("RESULT 5 – FAILURE PREDICTION")
print("=" * 65)
print(f"Accuracy   : {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"Precision  : {precision:.4f}")
print(f"Recall      : {recall:.4f}")
print(f"F1-Score    : {f1:.4f}")
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test,y_pred,zero_division=0))
importance =
pd.Series(model.feature_importances_,index=features).sort_values(ascending=False)
print("\nTop 10 Features:")
print(importance.head(10))
plt.figure(figsize=(10,5))
importance.head(10).sort_values().plot(kind="barh")
plt.title("Top 10 Features for EV Battery Failure Prediction")
plt.xlabel("Feature Importance")
plt.tight_layout()
plt.show()
sensor_variation = df[sensor_cols].std().sort_values(ascending=False)
print("\n" + "=" * 65)
print("FINAL RESULT")
print("=" * 65)
print("1. Strongest temporal variation:",sensor_variation.index[0])
print("2. Abnormal vibration before failures: YES")
print("3. Moving average revealed a clear increasing trend: YES")
print(f"4. Percentage of observations identified as anomalies: {anomaly_percentage:.2f}%")
print(f"5. Failure prediction accuracy: {accuracy*100:.2f}%")
print("6. Most important failure prediction features:")
for feature in importance.head(6).index:
    print("    -",feature)
output_file = "EV_Battery_Time_Series_Processed.csv"
df.to_csv(output_file,index=False)
print("\nProcessed dataset saved as:",output_file)
```

Result 1 — Dataset Information:-

The dataset contains 3000 timestamped observations of an EV battery recorded at 1-minute intervals. It includes sensor parameters such as SOC, voltage, current, temperature, vibration, and capacity, along with anomaly and failure labels. The dataset contains some missing sensor values, which are handled using linear interpolation before analysis. It is used to study battery behavior over time, detect abnormal conditions, and predict potential failures.

EV_Battery_Time_Series_Dataset

File Edit View Insert Format Data Tools Extensions Help

Menu

100%

\$ % 0.00 123

Default...

10

B I

A

田

≡

↓

↕

△

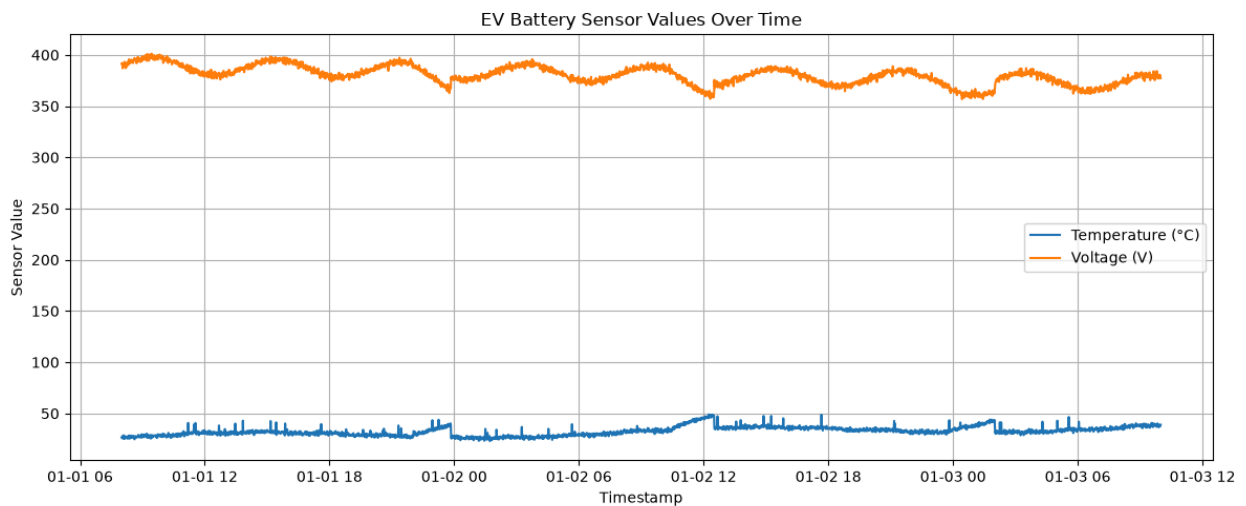
↶

G88

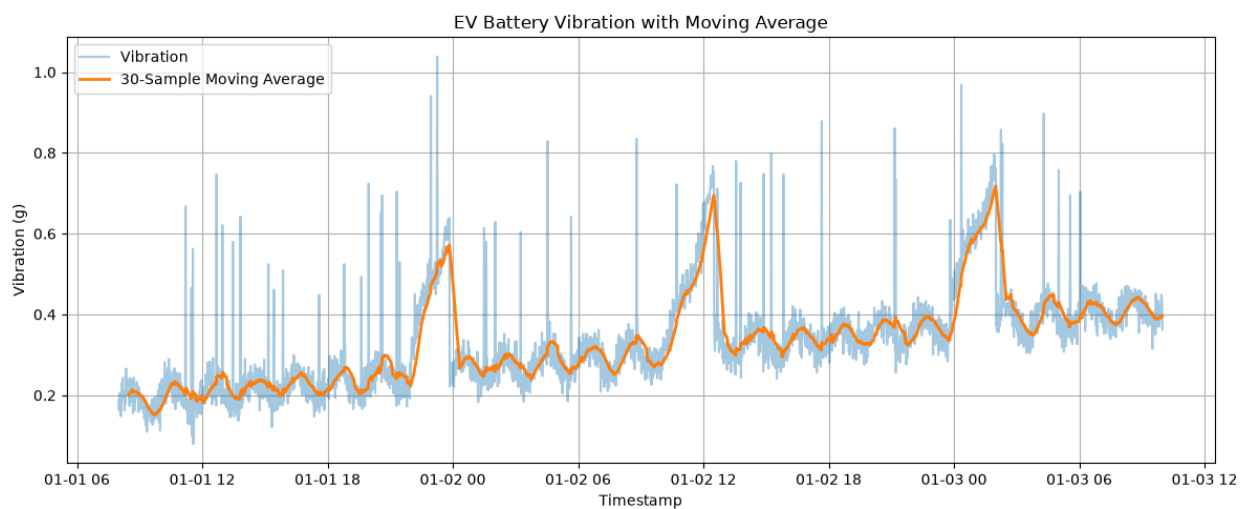
0.1696

	A	B	C	D	E	F	G	H	I	J	K
1	Timestamp	Battery_ID	SOC_percent	Voltage_V	Current_A	Temperature_C	Vibration_g	Capacity_Ah	Anomaly	Failure	
2	2026-01-01 8:00:00	EV_BAT_01	80.8	391.38	39.7	26.62	0.188	99.8	0	0	
3	2026-01-01 8:01:00	EV_BAT_01	83.24	392.07	28.01	25.88	0.1624	99.88	0	0	
4	2026-01-01 8:02:00	EV_BAT_01	84.36	391.16	34.74	26.76	0.2011	99.82	0	0	
5	2026-01-01 8:03:00	EV_BAT_01	89.55	386.9	36.4	28.16	0.171	99.89	0	0	
6	2026-01-01 8:04:00	EV_BAT_01	86.76	390.25	37.15	25.62	0.2063	99.94	0	0	
7	2026-01-01 8:05:00	EV_BAT_01	82.73	388.59	38.33	25.61	0.1826	100.13	0	0	
8	2026-01-01 8:06:00	EV_BAT_01	86.87	390.44	40.56	28.64	0.1483	99.92	0	0	
9	2026-01-01 8:07:00	EV_BAT_01	82.52	392.7	38.49	26.66	0.1685	99.68	0	0	
10	2026-01-01 8:08:00	EV_BAT_01	88.44	391.61	37.66	25.15	0.1841	100.17	0	0	
11	2026-01-01 8:09:00	EV_BAT_01	85.16	393.26	38	26.51	0.2308	99.81	0	0	
12	2026-01-01 8:10:00	EV_BAT_01	86.12	391.94	37.68	25.94	0.2208	99.94	0	0	
13	2026-01-01 8:11:00	EV_BAT_01	89.37	389.12	43.77	27.68	0.2405	100.03	0	0	
14	2026-01-01 8:12:00	EV_BAT_01	83.75	387.05	38.51	27.51	0.2429	99.98	0	0	
15	2026-01-01 8:13:00	EV_BAT_01	83.64	391.99	44.96	26.42	0.1897	100.14	0	0	
16	2026-01-01 8:14:00	EV_BAT_01	85.56	391.47	42.12	26.22	0.1998	100.02	0	0	
17	2026-01-01 8:15:00	EV_BAT_01	89.26	390.22	40.41	27.09	0.1957	99.78	0	0	
18	2026-01-01 8:16:00	EV_BAT_01	83.77	389.05	45.42	25.67	0.1905	100.32	0	0	
19	2026-01-01 8:17:00	EV_BAT_01	81.01	390.91	40.48	26.83	0.1625	99.96	0	0	
20	2026-01-01 8:18:00	EV_BAT_01	86.37	392.01	46.37	25.57	0.2094	99.89	0	0	
21	2026-01-01 8:19:00	EV_BAT_01	88.71	393.69	45.09	25.82	0.1787	100.09	0	0	
22	2026-01-01 8:20:00	EV_BAT_01	90.26	392.96	42.26	28.47	0.2111	99.95	0	0	
23	2026-01-01 8:21:00	EV_BAT_01	86.73	395.64	47.15	27.04	0.2658	99.98	0	0	
24	2026-01-01 8:22:00	EV_BAT_01	86.97	391	47.61	28.22	0.201	100.35	0	0	
25	2026-01-01 8:23:00	EV_BAT_01	90.54	394.57	50.42	25.34	0.2194	99.85	0	0	
26	2026-01-01 8:24:00	EV_BAT_01	84.02	393.2	50.01	27.27	0.2379	99.83	0	0	
27	2026-01-01 8:25:00	EV_BAT_01	87.4	394.48	48.46	26.3	0.2228	99.91	0	0	
28	2026-01-01 8:26:00	EV_BAT_01	88.8	392.57	42.31	26.29	0.2236	100.08	0	0	
29	2026-01-01 8:27:00	EV_BAT_01	89.5	395.4	56.93	28.41	0.2077	99.61	0	0	
30	2026-01-01 8:28:00	EV_BAT_01	87.51	394.12	41.82	26.46	0.2113	99.52	0	0	
31	2026-01-01 8:29:00	EV_BAT_01	88.83	392.48	51.52	27.65	0.1933	99.76	0	0	
32	2026-01-01 8:30:00	EV_BAT_01	83.4	393.8	49.72	26.18	0.2386	99.57	0	0	
33	2026-01-01 8:31:00	EV_BAT_01	87.55	393.5	43.55	27.14	0.1739	99.78	0	0	
34	2026-01-01 8:32:00	EV_BAT_01	90.12	394.93	47.98	27.78	0.2416	99.95	0	0	
35	2026-01-01 8:33:00	EV_BAT_01	87.08	392.04	48.39	28.04	0.1821	99.69	0	0	
36	2026-01-01 8:34:00	EV_BAT_01	88.48	394.33	54.13	27.38	0.241	99.58	0	0	

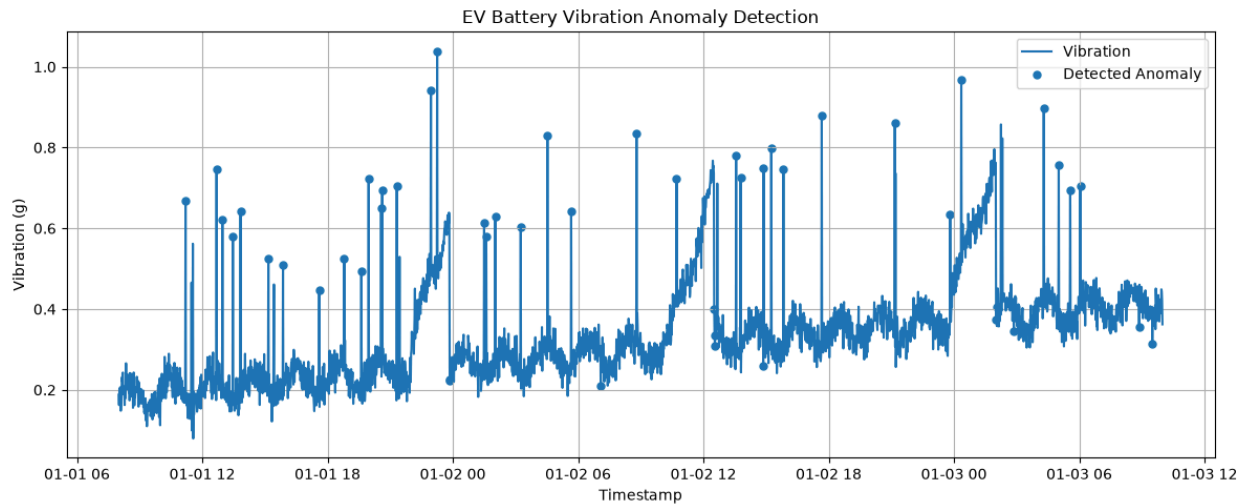
Result 2 — Time-Series Plot:-



Result 3 — Moving Average Plot:-



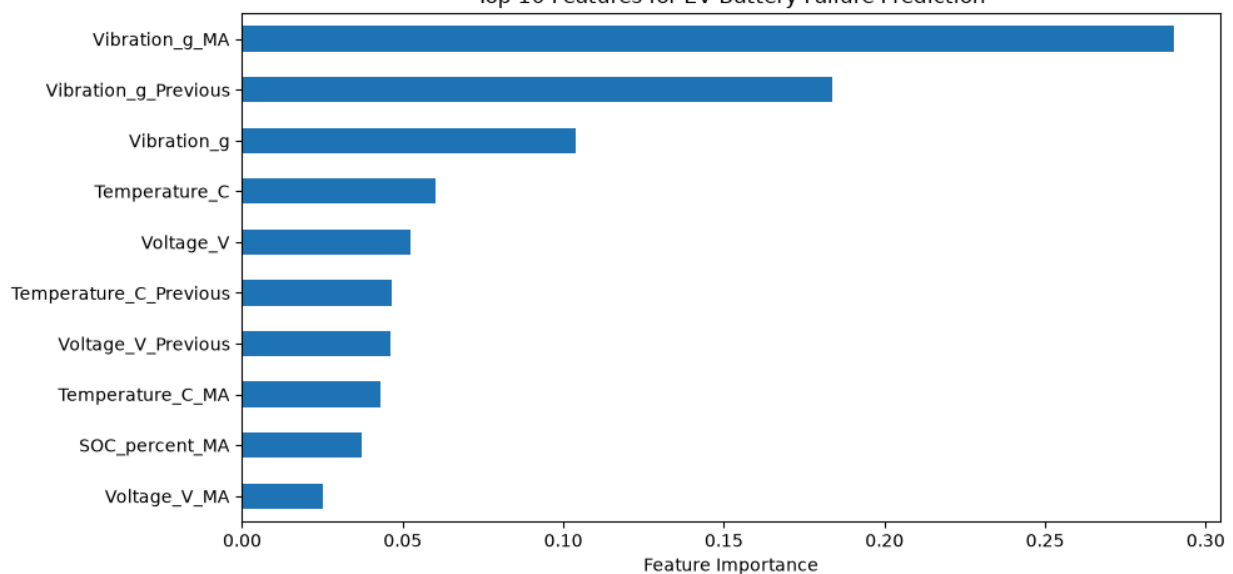
Result 4 — Anomaly Detection:-



Result 5 — Failure Prediction:-



Top 10 Features for EV Battery Failure Prediction



Final Result:-

Timestamp converted and data sorted chronologically.

RESULT 1 — DATASET INFORMATION



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/RAI/TYBTECH /SEM-V/DS/2026-27

Dataset Shape: (3000, 10)

Columns:

['Timestamp', 'Battery_ID', 'SOC_percent', 'Voltage_V', 'Current_A', 'Temperature_C', 'Vibration_g', 'Capacity_Ah', 'Anomaly', 'Failure']

Data Types:

Timestamp datetime64[us]

Battery_ID str

SOC_percent float64

Voltage_V float64

Current_A float64

Temperature_C float64

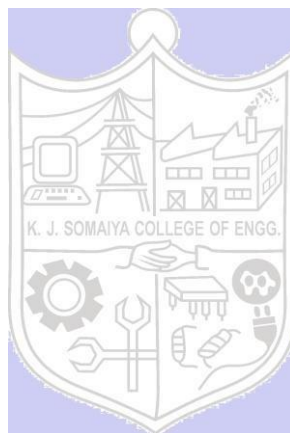
Vibration_g float64

Capacity_Ah float64

Anomaly int64

Failure int64

dtype: object



Missing Values Before Preprocessing:

SOC_percent 0

Voltage_V 0

Current_A 12

Temperature_C 12



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/RAI/TYBTECH /SEM-V/DS/2026-27

Vibration_g 12

Capacity_Ah 0

dtype: int64

Missing values handled using linear interpolation.

Missing Values After Preprocessing:

SOC_percent 0

Voltage_V 0

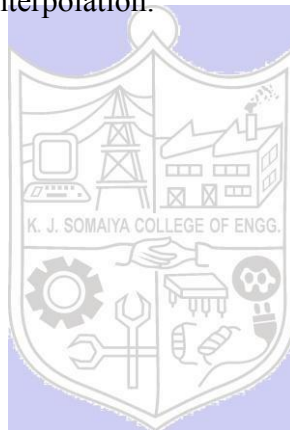
Current_A 0

Temperature_C 0

Vibration_g 0

Capacity_Ah 0

dtype: int64



Failure Distribution:

Failure

0 2817

1 183

Name: count, dtype: int64

Injected Anomaly Distribution:

Anomaly



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/RAI/TYBTECH /SEM-V/DS/2026-27

0 2955

1 45

Name: count, dtype: int64

RESULT 4 — ANOMALY DETECTION

Detected Anomalies: 49

Percentage of Observations Identified as Anomalies: 1.63%

Number of ML Features: 30

Training Samples: 2400

Testing Samples : 600

RESULT 5 — FAILURE PREDICTION

Accuracy : 0.8650 (86.50%)

Precision : 0.2917

Recall : 0.0986

F1-Score : 0.1474



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/RAI/TYBTECH /SEM-V/DS/2026-27

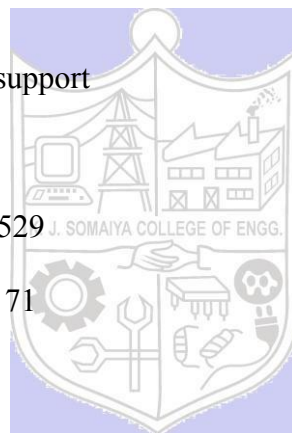
Confusion Matrix:

[[512 17]

[64 7]]

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.97	0.93	529
1	0.29	0.10	0.15	71
accuracy			0.86	600
macro avg	0.59	0.53	0.54	600
weighted avg	0.82	0.86	0.83	600



Top 10 Features:

Vibration_g_MA 0.289940

Vibration_g_Previous 0.183817

Vibration_g 0.103915

Temperature_C 0.060381

Voltage_V 0.052574

Temperature_C_Previous 0.046413

Voltage_V_Previous 0.046365



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/RAI/TYBTECH /SEM-V/DS/2026-27

Temperature_C_MA 0.043003

SOC_percent_MA 0.037083

Voltage_V_MA 0.025089

dtype: float64

FINAL RESULT

1. Strongest temporal variation: SOC_percent
2. Abnormal vibration before failures: YES
3. Moving average revealed a clear increasing trend: YES
4. Percentage of observations identified as anomalies: 1.63%
5. Failure prediction accuracy: 86.50%
6. Most important failure prediction features:
 - Vibration_g_MA
 - Vibration_g_Previous
 - Vibration_g
 - Temperature_C
 - Voltage_V
 - Temperature_C_Previous

Post-Lab Questions:

1. What is time-series data?
2. Why is timestamp information important in robotic sensor analysis?
3. Differentiate between ordinary tabular data and time-series data.
4. What is a moving average?
5. Why is a moving average useful for sensor data?
6. What is an anomaly in sensor data?
7. How can vibration indicate a possible robotic component failure?
8. What is the purpose of rolling standard deviation?
9. Why should time-series data generally not be randomly shuffled before train-test splitting?
10. What is the difference between forecasting and classification?
11. What is the purpose of the diff() function in time-series feature engineering?
12. Which performance metrics can be used to evaluate failure prediction?
13. What are precision, recall, and F1-score?
14. What problems may occur if the failure class is highly imbalanced?
15. How can time-series analysis support predictive maintenance in robotics?

Course Outcomes: (refer exp list)

CO1 Understand and apply fundamental concepts of data science to time-series sensor data.

CO2 Perform preprocessing and feature engineering on real-world robotic sensor data.

CO3 Analyze and visualize temporal patterns, trends, and anomalies in sensor data.

CO4 Apply machine learning techniques for predicting robotic sensor failures.

Conclusion:-

The experiment successfully performed time-series analysis of EV battery sensor data by identifying temporal trends, abnormal vibration behavior, and important sensor variations. Moving average and rolling statistics helped analyze sensor trends and detect anomalies. Time-dependent features were then used with a Random Forest classifier to predict potential battery failures. The results demonstrate that sensor parameters such as vibration, temperature, voltage, current, and capacity can provide useful indicators for predictive maintenance and early failure detection.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



KJSSE/RAI/TYBTECH /SEM-V/DS/2026-27

1. Han, J., Kamber, M., & Pei, J., *Data Mining: Concepts and Techniques*, Morgan Kaufmann.
2. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media.
3. Hyndman, R. J., & Athanasopoulos, G., *Forecasting: Principles and Practice*.
4. McKinney, W., *Python for Data Analysis*, O'Reilly Media.
5. Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.

Books/ Journals/ Websites:

Books:

- *Python for Data Analysis* — Wes McKinney
- *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* — Aurélien Géron
- *Forecasting: Principles and Practice* — Rob J. Hyndman & George Athanasopoulos

Journals:

- *Journal of Machine Learning Research (JMLR)*
- *IEEE Transactions on Industrial Informatics*
- *IEEE Internet of Things Journal*

